



Tribhuvan University

Faculty of Humanities and Social Sciences

**A PROJECT
REPORT ON
PROPERTY MANAGEMENT SYSTEM**

*In partial fulfillment of the requirements for the degree of Bachelor of Computer
Applications*

**Submitted to Department of
Computer Application
Bhaktapur Multiple Campus**

Submitted by

Aayush Kakshapati (6-2-20-3-2022)

Under the Supervision of

Mr. Madan Nath



Tribhuvan University
Faculty of Humanities and Social Sciences
Bhaktapur Multiple Campus

SUPERVISOR'S RECOMMENDATION

I hereby recommend that this project prepared under my supervision by **AAYUSH KAKSHAPATI** entitled “**PROPERTY MANAGEMENT SYSTEM**” in partial fulfillment of the requirements for the degree of Bachelor of Computer Application is recommended for the final evaluation.

.....

Mr. Madan Nath

Supervisor

Lecturer, BCA Department

Bhaktapur Multiple Campus



Tribhuvan University

Faculty of Humanities and Social Sciences

Bhaktapur Multiple Campus

LETTER OF APPROVAL

This is to certify that this project prepared by **AAYUSH KAKSHAPATI** entitled **“PROPERTY MANAGEMENT SYSTEM”** in partial fulfillment of the requirements for the degree of Bachelor in Computer Application has been evaluated. In our opinion, it is satisfactory in the scope and quality as a project for the required degree.

.....

Mr. Madan Nath

Coordinator,

Bhaktapur Multiple Campus

.....

Mr. External Examiner

.....

Internal Examiner

.....

External Examiner

ABSTRACT

PropFlow is a web-based Property Management System designed to help property owners efficiently manage multiple rental units across different locations. In traditional property management practices, landlords often rely on manual records or fragmented digital tools to track tenants, rent payments, maintenance requests, and property availability. These methods are inefficient, error-prone, and become increasingly difficult to manage as the number of properties grows. PropFlow addresses these challenges by providing a centralized and structured digital platform that streamlines core property management operations. The system enables property owners to manage property listings, register and track tenants, monitor rent payments, handle maintenance requests, and view real-time availability of units. Additionally, it provides dashboards and reports to support better financial tracking and decision-making.

The system is built using a modern full-stack architecture, with Next.js for the frontend, Django for backend services, and PostgreSQL for database management. This combination ensures scalability, security, and efficient data handling while maintaining a responsive and user-friendly interface.

Keywords: *Property Management System, Tenant Management, Rent Tracking, Maintenance Management, Web Application, Dashboard System*

ACKNOWLEDGEMENT

I deeply appreciate the generous support and contributions of all those who helped us achieve success in our system endeavors. I would like to extend my heartfelt gratitude to my supervisor Mr. Madan Nath for his invaluable guidance and support throughout my project. His expertise, dedication, valuable encouragement and friendly environment with me have played a vital role in shaping the successful completion of this project. I am also thankful to my faculty members and institution for providing the necessary resources and learning environment to successfully complete this project. Finally, I would like to extend my appreciation to my friends and peers for their support, ideas, and motivation during the development process.

With Regards:

Aayush Kakshapati

TABLE OF CONTENTS

ABSTRACT.....	i
ACKNOWLEDGMENT.....	ii
TABLE OF CONTENTS.....	iii
LIST OF ABBREVIATIONS.....	iv
LIST OF FIGURES.....	v
LIST OF TABLES.....	vi
CHAPTER 1: INTRODUCTION.....	1
1.1 Introduction.....	1
1.2 Problem Statement.....	1
1.3 Objectives.....	2
1.4 Scope And Limitation.....	2
1.4.1 Scope.....	2
1.4.2 Limitations.....	2
1.5 Development Methodology.....	3
1.6 Report Organization.....	4
CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW.....	5
2.1 Background Study.....	5
2.2 Literature Review.....	5
CHAPTER 3: SYSTEM ANALYSIS AND DESIGN.....	7
3.1 System Analysis.....	7
3.1.1 Requirement analysis.....	7
3.1.2 Feasibility analysis.....	9
3.1.3 Data Modeling.....	10
3.1.4 Process Modeling.....	10
3.2 System Design.....	11
3.2.1 Architectural Design.....	11

3.2.2 Process Modeling (DFD).....	12
3.2.3 Database Schema Design.....	13
3.3 Interface Design (UI Interface).....	14
3.4 Algorithm Used.....	15
3.3.1 Late Fee Calculation Algorithm.....	16
3.3.2 Occupancy Rate Calculation Algorithm.....	17
3.3.3 Maintenance Request Prioritization Algorithm.....	18
3.5 Result Analysis.....	19
CHAPTER 4: IMPLEMENTATION AND TESTING.....	21
4.1 Implementation.....	21
4.1.1 Tools Used.....	21
4.1.2 Implementation Details of Modules.....	20
4.2 Testing.....	26
4.2.1 Test cases for Unit Testing.....	26
4.2.2 Test Cases for System Testing.....	27
CHAPTER 5: CONCLUSIONS AND FUTURE RECOMMENDATIONS.....	28
5.1 Lesson Learnt/Outcome.....	28
5.2 Conclusion.....	28
5.3 Future recommendations.....	29
REFERENCES.....	30
APPENDIX.....	31

LIST OF ABBREVIATIONS

CASE	Computer Aided Software Engineering
DFD	Data Flow Diagram
ERD	Entity Relation Diagram
ICT	Information Technology
INFO	Information
IDE	Integrated Development Environment
SDLC	Software Development Life Cycle
UI	User Interface

LIST OF FIGURES

Figure 1.1 Hybrid I Model.....	3
Figure 3.1: Use Case Diagram of PROPERTY MANAGEMENT SYSTEM.....	9
Figure 3.2: Class Diagram of PROPERTY MANAGEMENT SYSTEM.....	13
Figure 3.3: Sequence Diagram of PROPERTY MANAGEMENT SYSTEM.....	13
Figure 3.4: Architectural Design of PROPERTY MANAGEMENT SYSTEM.....	13
Figure 3.5: Level 0 DFD of PROPERTY MANAGEMENT SYSTEM.....	14
Figure 3.6: Database Schema Design of PROPERTY MANAGEMENT SYSTEM.....	15
Figure 3.7: User login page UI.....	16
Figure 3.8: Admin Login Page UI.....	16
Figure 3.9: Admin Dashboard.....	17
Figure 3.10: User Dashboard.....	17

LIST OF TABLES

Table 4.1: Test Case for Login.....	22
Table 4.2: Test Cases for Sign Up.....	22
Table 4.3: Test Cases for Creating Property.....	23
Table 4.4 Test Cases for system testing of the PROPERTY MANAGEMENT SYSTEM.....	23

CHAPTER-1 INTRODUCTION

1.1 Introduction

The growth of rental properties and urban housing has increased the need for efficient property management solutions. Property owners who manage multiple units across different locations often deal with large amounts of information, including tenant details, rent payments, maintenance activities, and property availability.

Traditionally, these tasks are handled using manual methods such as paper records or basic digital tools like spreadsheets. While these methods may work for small-scale operations, they lack structure, scalability, and real-time accessibility.

To improve efficiency and organization, digital solutions are increasingly being adopted in property management. A centralized system can help property owners manage their operations more effectively by providing structured data handling, real-time updates, and better control over their properties. This project introduces PropFlow, a web-based Property Management System designed to assist property owners in managing multiple properties and units in a simple and organized manner. The system focuses on improving visibility, reducing manual effort, and providing better control over property-related operations.

1.2 Problem Statement

Despite the availability of digital tools, many property owners still rely on manual or semi-digital methods to manage their properties. This creates several operational challenges, especially when handling multiple units across different locations. One major issue is the lack of a centralized system for tracking rent payments, which often leads to inconsistent records and difficulty in monitoring income. Tenant information is not properly organized, making it hard to manage and retrieve when needed. Maintenance requests are usually handled informally, resulting in delays, missed issues, and lack of proper tracking. Additionally, property owners do not have real-time visibility into unit availability, which affects occupancy management. These challenges lead to inefficiencies, increased workload, and reduced control over property operations. Therefore, there is a need for a structured and centralized system like PropFlow that can simplify property management, improve data organization, and enhance operational efficiency.

1.3 Objectives

The main objective of PropFlow is to simplify and organize property management for owners handling multiple units across different locations.

Specific objectives include:

- To provide a centralized system for tracking property occupancy, tenant details, and rent status in real time.
- To provide payment and maintenance tracking between tenants and property owners.

1.4 Scope and Limitation

1.4.1 Scope

PropFlow is a web-based system designed to improve the daily operations of property owners by streamlining key property management tasks. The system enables tenant registration and association with specific rental units, supports rent tracking and payment status monitoring, and allows tenants to submit maintenance requests while enabling owners to track their resolution progress. Additionally, PropFlow provides occupancy management through availability tracking of vacant and occupied units, along with basic reporting and dashboard visualization features to help property owners monitor overall property performance effectively.

1.4.2 Limitation

The system does not integrate with banking or automated payment gateways.

- Mobile application support is not included in the current version.
- Advanced analytics and AI-based predictions are not implemented.
- Offline functionality is not supported.
- Real-time notification is not supported.
- Scalability for large enterprise-level property networks is limited without future upgrades.
- Legal, taxation, and regulatory compliance modules are not included

1.5 Development Methodology

The development of PropFlow follows a Hybrid Incremental Model, which combines structured planning with step-by-step system implementation. This approach allows the system to be developed in manageable modules while maintaining overall design consistency.

The process begins with requirement analysis, where the core functionalities such as property management, tenant handling, rent tracking, maintenance management, and reporting are identified. This is followed by system design, including database structure, system architecture, and user interface planning. The implementation phase is carried out incrementally. Each module—such as property listing, tenant management, payment tracking, and maintenance handling—is developed, tested, and integrated independently. This modular approach reduces complexity and allows early validation of system functionality.

Testing is performed throughout the development process, including unit testing for individual components and integration testing to ensure smooth interaction between modules. Regular feedback and revisions are incorporated to improve system performance and usability.

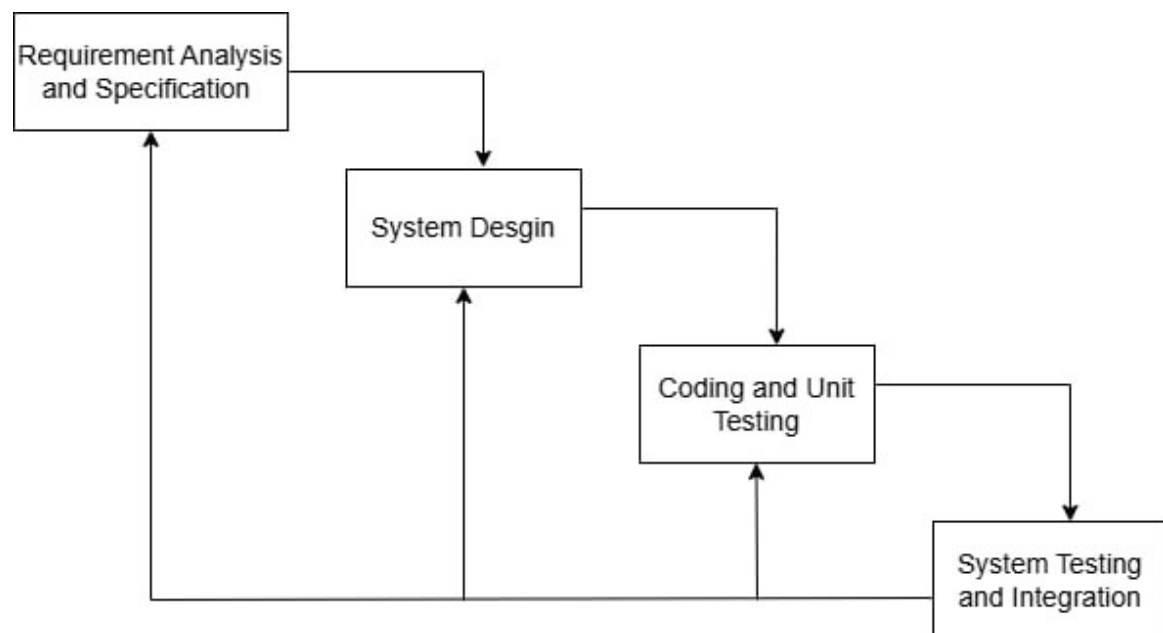


Figure 1.1: Hybrid Waterfall Model

1.6 Report Organization

This report is structured into five chapters, each focusing on a specific aspect of the PropFlow system.

Chapter 1: Introduces the project, outlining the background, problem statement, objectives, scope, limitations, and development methodology. It sets the foundation for understanding the purpose and direction of the system.

Chapter 2: Presents the background study and literature review, covering key concepts related to property management systems and reviewing similar existing solutions.

Chapter 3: Focuses on system analysis and design. It includes requirement analysis, feasibility study, and system design components such as ER diagrams, data flow diagrams (DFD), system architecture, and database schema.

Chapter 4: Describes the implementation and testing of the system. It explains the technologies used, module-wise development, and different testing approaches to ensure system reliability.

Chapter 5: Concludes the report by summarizing the overall project, highlighting key outcomes, and providing recommendations for future improvements.

CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

Property management involves the administration of residential or commercial real estate, including handling tenants, rent collection, maintenance, and property occupancy. With the growth of urban housing and rental markets, property owners increasingly manage multiple units across different locations, making traditional management approaches less effective.

In earlier practices, property management tasks were handled manually using paper records or basic tools such as spreadsheets. While these methods provide basic record-keeping, they lack real-time updates, data consistency, and centralized access. As the volume of data increases, manual systems become inefficient, leading to errors in rent tracking, difficulty in managing tenant records, and delays in handling maintenance issues.

Modern property management systems aim to address these limitations by digitizing operations and providing a centralized platform for managing all aspects of property administration. These systems typically include features such as property listing, tenant management, payment tracking, maintenance handling, and reporting.

With advancements in web technologies, full-stack applications using frameworks such as Next.js and Django, along with relational databases like PostgreSQL, enable the development of scalable and secure systems. These technologies support real-time data handling, improved user experience, and efficient backend processing.

The development of PropFlow is based on these concepts, aiming to provide a structured and centralized system tailored for individual property owners managing multiple rental units.

2.2 Literature Review

Several studies and existing systems have explored the use of digital platforms to improve property and rental management processes. These works highlight the importance of centralized data management, automation, and usability in reducing operational inefficiencies.

According to [1], traditional property management practices rely heavily on manual record-keeping, which leads to inconsistencies in tenant data, rent tracking, and

maintenance management. The study emphasizes that digitized systems significantly improve accuracy and reduce administrative workload.

Research conducted in [2] discusses the role of web-based management systems in handling real estate operations. The study shows that integrating modules such as tenant management, payment tracking, and reporting into a single platform improves transparency and decision-making for property owners.

In [3], the authors highlight the importance of user-friendly interfaces in management systems, particularly for non-technical users such as individual landlords. The study suggests that systems with simple dashboards and clear data visualization improve usability and adoption rates.

Another study [4] focuses on database-driven applications and emphasizes that structured relational databases enhance data consistency and enable efficient querying of large datasets. This is particularly important in systems where multiple properties, tenants, and transactions are involved.

Furthermore, [5] discusses the use of modern web technologies in building scalable applications. The study notes that frameworks such as modern JavaScript-based frontends and backend frameworks improve performance, maintainability, and scalability of web systems.

Despite these advancements, many existing systems are either too complex or not tailored for small-to-medium property owners. PropFlow addresses this gap by providing a simplified yet structured system focused on essential property management functionalities, ensuring ease of use while maintaining efficiency.

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 System Analysis

The System Analysis for the Property Management System involves understanding the system requirements, analyzing its functionality, and identifying the challenges to be addressed. This section includes a detailed exploration of the functional and non-functional requirements, and feasibility analysis.

3.1.1 Requirement Analysis

i. Functional Requirements

The functional requirements of PropFlow describe the core operations of the system:

1. Property Management

- Add, update, and delete properties and units
- View property details and status

2. Tenant Management

- Register tenant information
- Assign tenants to specific units
- View and manage tenant records

3. Rent Management

- Record rent payments
- Track payment status (paid/unpaid)
- Maintain payment history

4. Maintenance Management

- Submit maintenance requests
- Track request status (pending/resolved)
- Manage maintenance records

5. Availability Tracking

- View occupied and vacant units
- Update unit availability status

6. Dashboard & Reports

- Display summary of properties, tenants, and payments
- Generate simple reports for analysis

3.1: Use Case Diagram of PROPERTY MANAGEMENT SYSTEM

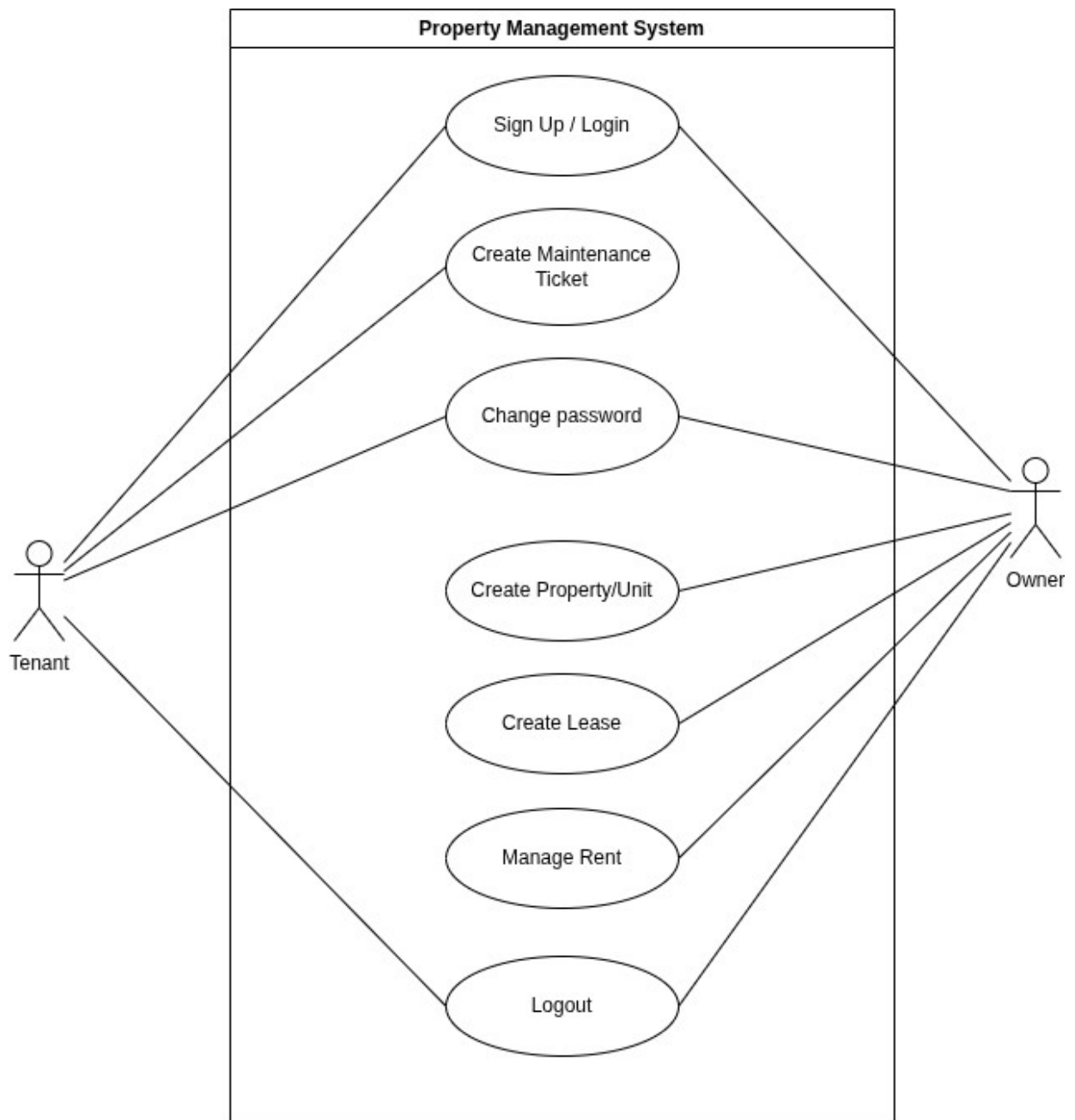


Figure 3.1: Use Case Diagram of PROPERTY MANAGEMENT SYSTEM

ii. Non-Functional Requirements

These define system quality and performance:

1. Usability

- Simple and intuitive interface for property owners

2. Performance

- Fast response time for data retrieval and updates

3. Security

- User authentication and authorization
- Protection of sensitive data

4. Scalability

- Ability to handle increasing number of properties and users

5. Reliability

- Accurate data storage and minimal system downtime

6. Maintainability

- Modular code structure for easy updates and improvements

3.1.2 Feasibility Analysis

A comprehensive feasibility analysis was conducted to evaluate the practicality and viability of developing the Property Management System. The evaluation considers technical, economic, operational, and schedule feasibility, ensuring that the system can be implemented efficiently while ensuring that the project is achievable, cost-effective, and operationally efficient within the planned timeline.

a. Technical Feasibility

PropFlow is technically feasible as it uses modern and widely adopted technologies such as Next.js, Django, and PostgreSQL. These technologies support scalable web applications and efficient data handling. The required development tools and resources are readily available.

b. Economic Feasibility

The system is cost-effective since it uses open-source technologies. Development costs are minimal, making it suitable for academic and small-scale deployment without requiring significant financial investment.

c. Operational Feasibility

The system is easy to use and designed for property owners with basic technical knowledge. It simplifies existing manual processes, making it practical for real-world usage.

d. Schedule Feasibility

The project can be completed within the given academic timeframe using incremental development. Each module is implemented step-by-step, ensuring steady progress.

3.1.3 Data Modeling

The Class diagram of the system is given below:

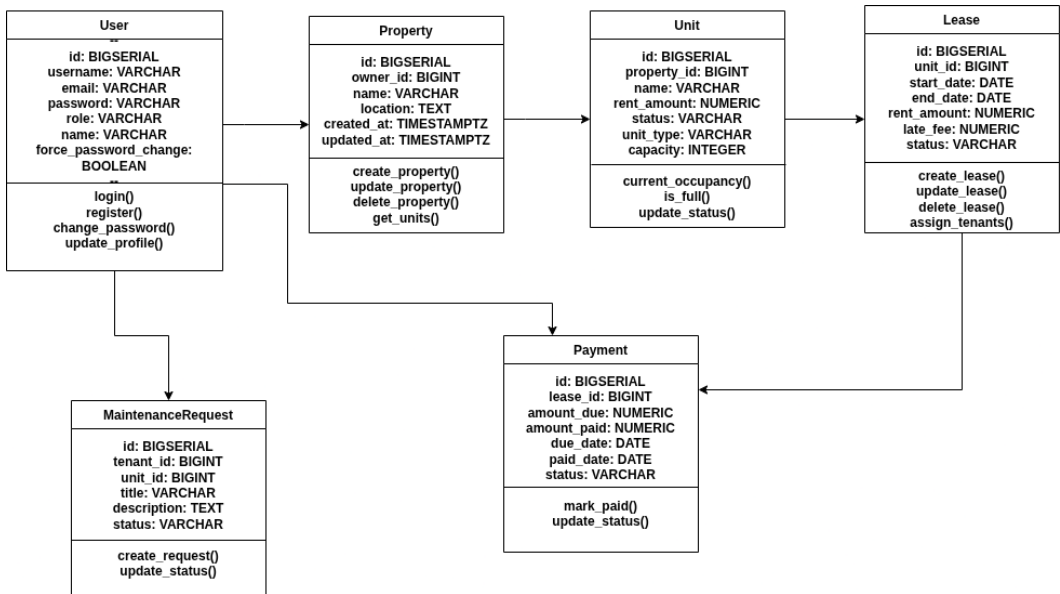


Figure 3.2: Class Diagram of PROPERTY MANAGEMENT SYSTEM

3.1.4 Process Modeling

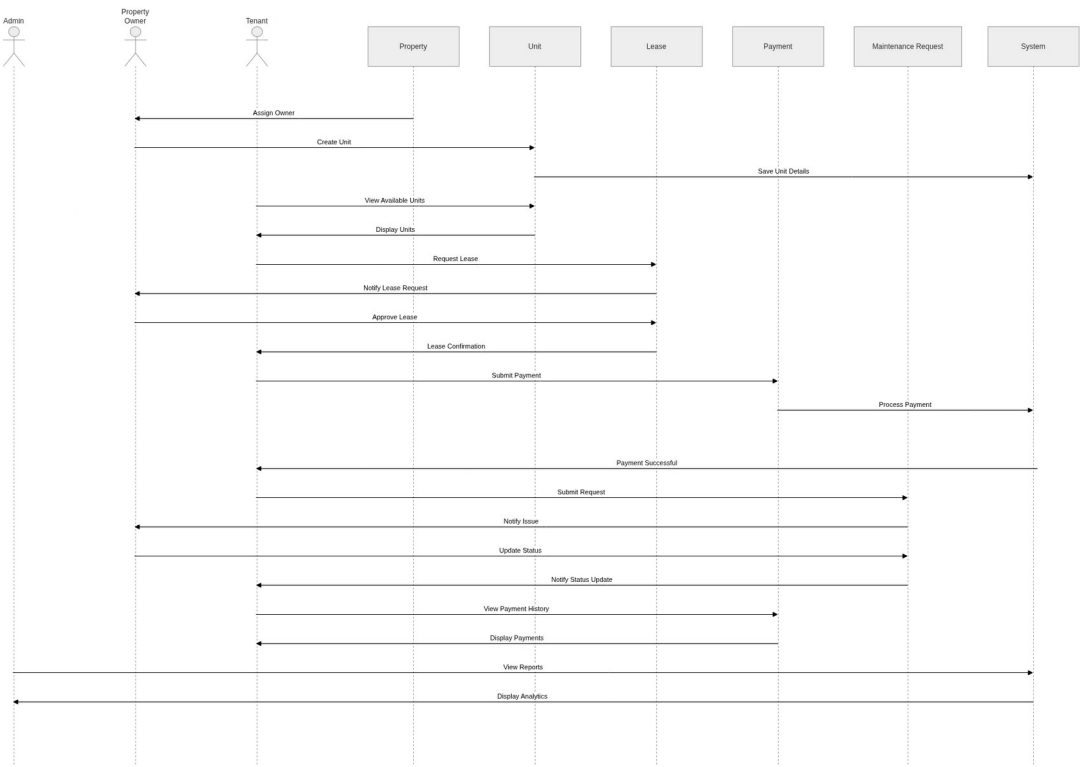


Figure 3.3: Sequence Diagram of PROPERTY MANAGEMENT SYSTEM

3.2 System Design

The system design of 'Property Management System' consists of architectural design, database schema design, user interface design and physical DFD are shown as follows:

3.2.1 Architectural design

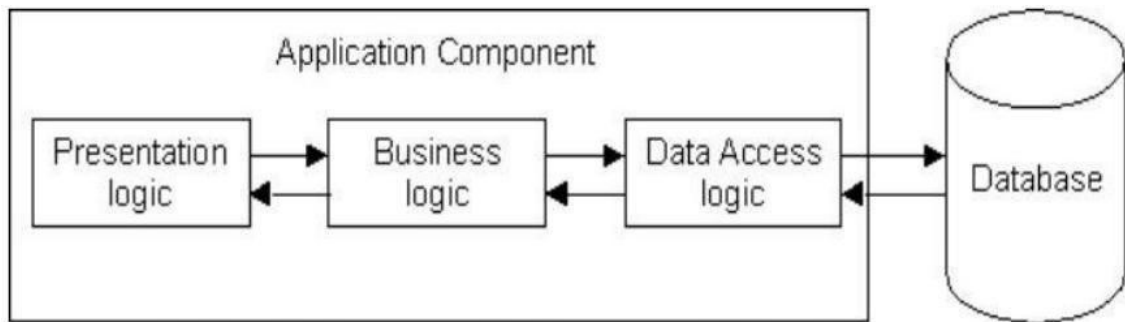


Figure 3.4: Architectural Design of PROPERTY MANAGEMENT SYSTEM

3.2.2 Process Modeling (DFD)

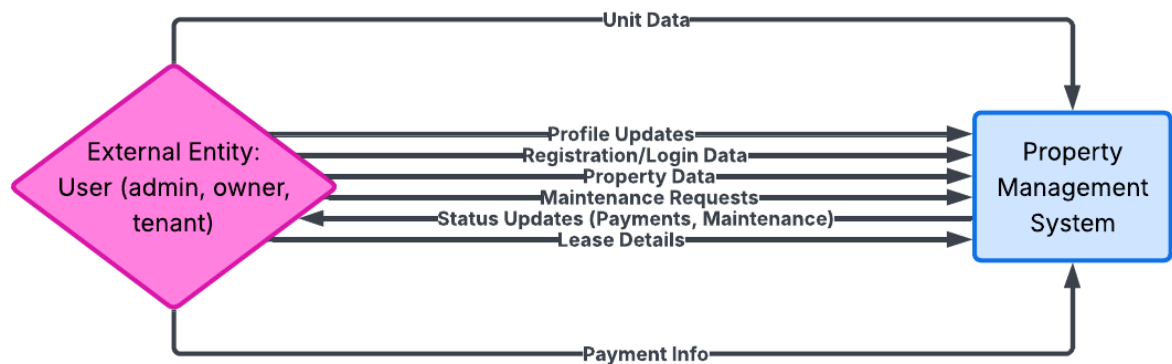


Figure 3.5: Level 0 DFD of PROPERTY MANAGEMENT SYSTEM

3.2.3 Database Schema Design

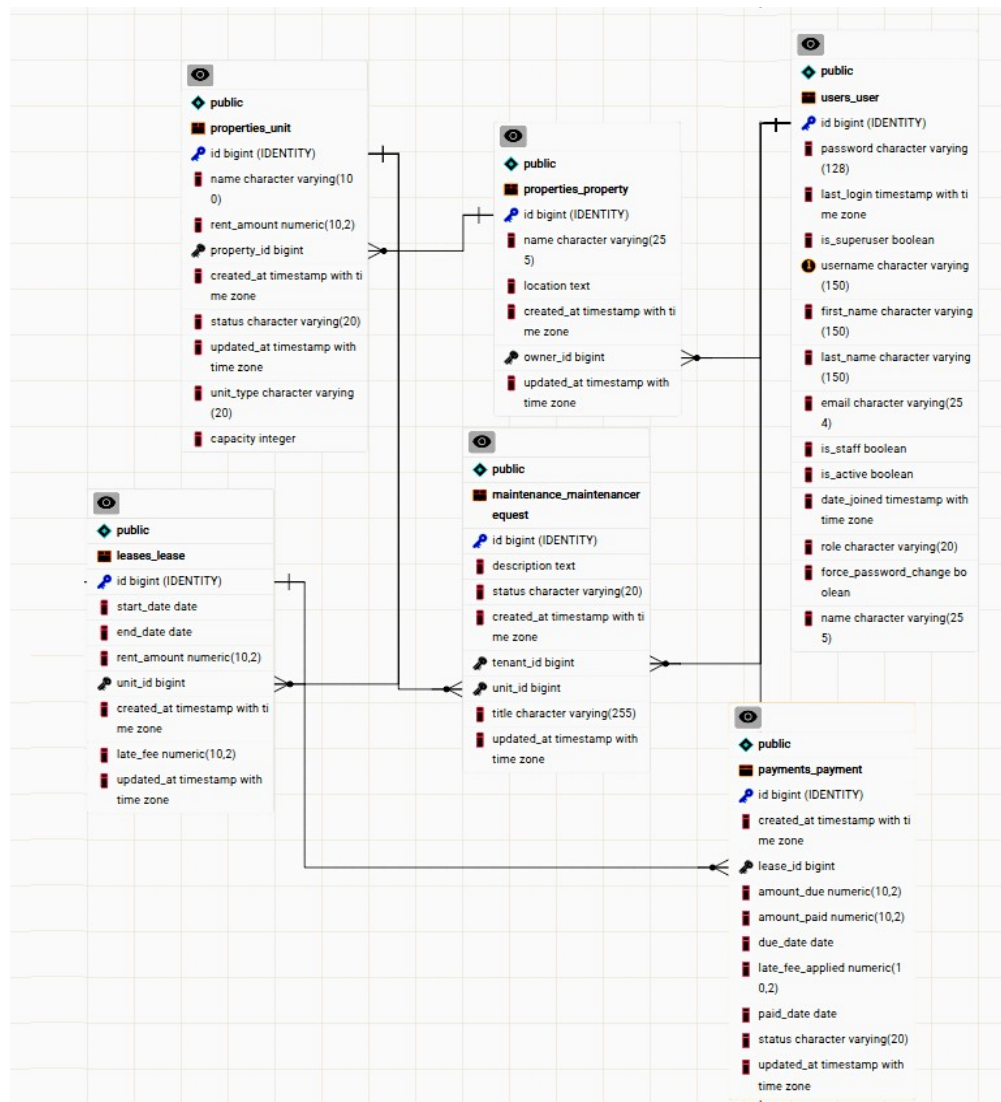


Figure 3.6: Database Schema Design of PROPERTY MANAGEMENT SYSTEM

3.3 Interface Design (UI Interface)

The interface design for the major pages of Property Management System are shown as follows:

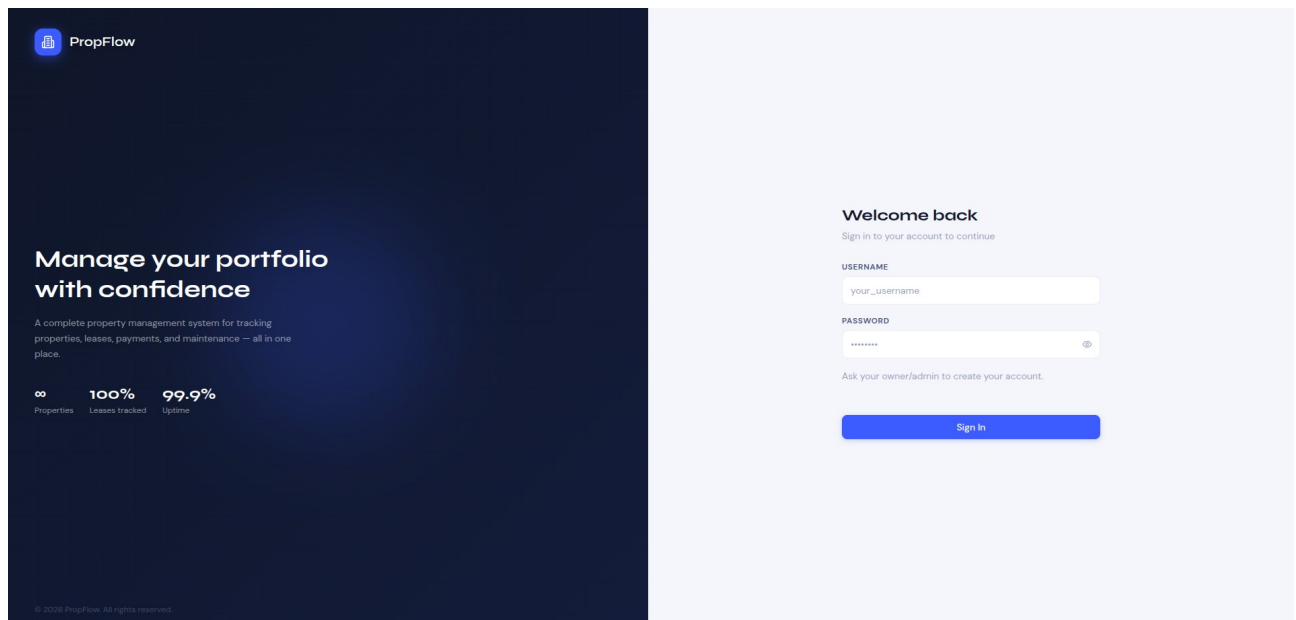


Figure 3.7: User login page UI

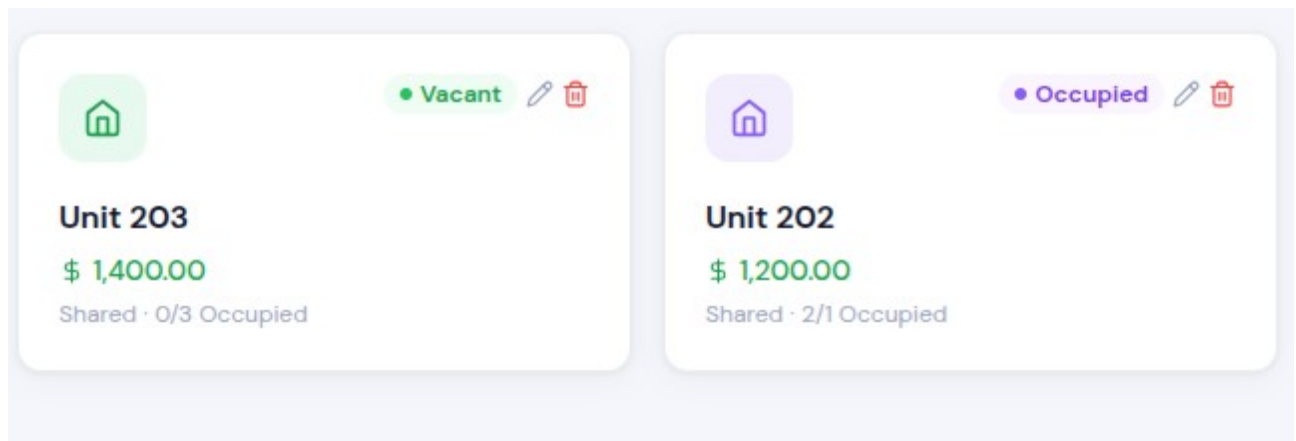


Figure 3.8: Unit Page UI

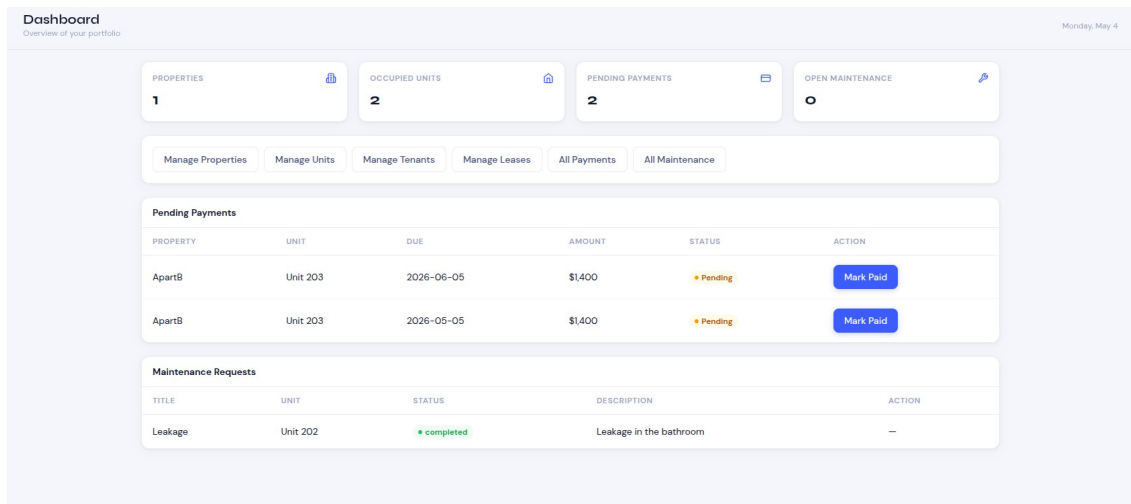


Figure 3.9: Admin Dashboard

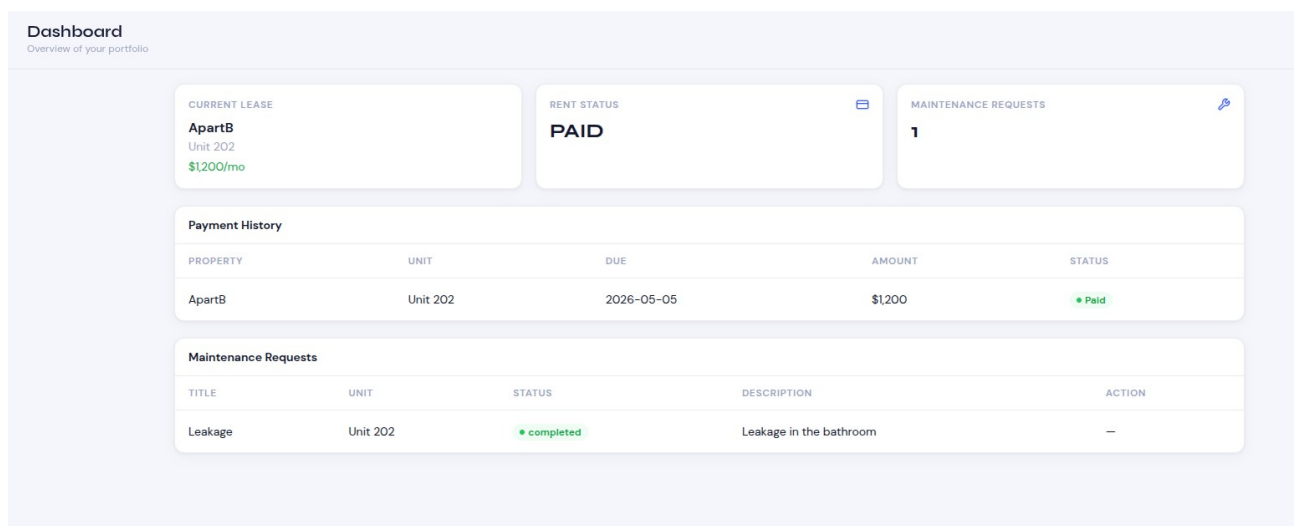


Figure 3.10: User Dashboard

3.4 Algorithm Used

The PROPERTY MANAGEMENT SYSTEM employs a combination of algorithmic techniques designed to optimize vehicle retrieval, sorting, and personalized recommendations, enhancing both system performance and user experience.

3.3.1 Late Fee Calculation Algorithm

In this system, the late fee calculation algorithm is used to determine penalty fees for rent payments that are submitted after the due date. The algorithm uses a tiered structure where the penalty percentage increases based on how many days the payment is overdue. This ensures fair penalty calculation while encouraging timely payments.

The steps involved in the implementation of the algorithm are:

Step 1: Take the rent amount, due date, and payment date as input.

Eg: `rent_amount = 1000, due_date = 2024-01-15, payment_date = 2024-01-25`

Step 2: Check if payment_date is null, if so, use today's date.

Eg: `payment_date = 2024-01-25 (already provided)`

Step 3: Compare payment_date with due_date to determine if payment is late.

Eg: `2024-01-25 > 2024-01-15 → Payment is late`

Step 4: If payment is not late or on time, return 0 as late fee.

Eg: `If payment_date <= due_date, late_fee = 0`

Step 5: Calculate days overdue by subtracting due_date from payment_date.

Eg: `days_overdue = 2024-01-25 - 2024-01-15 = 10 days`

Step 6: Apply tiered fee structure based on days overdue:

- If `days_overdue ≤ 5`: 5% of `rent_amount`
- If `days_overdue ≤ 15`: 10% of `rent_amount`
- If `days_overdue > 15`: 15% of `rent_amount`

Eg: `days_overdue = 10, so fee_percentage = 10% (0.10)`

Step 7: Calculate late fee by multiplying `rent_amount` with `fee_percentage`.

Eg: `late_fee = 1000 × 0.10 = 100`

Step 8: Round the result to 2 decimal places for currency formatting.

Eg: `late_fee = 100.00`

Step 9: Return the calculated late fee.

Eg: `Final late_fee = 100.00`

3.3.2 Occupancy Rate Calculation Algorithm

In this system, the occupancy rate calculation algorithm is used to determine the percentage of occupied units across all properties. This metric helps property owners and administrators understand property utilization and make informed decisions about rental strategies. The algorithm also categorizes the occupancy status into High, Medium, or Low levels for quick assessment.

The steps involved in the implementation of the algorithm are:

Step 1: Retrieve all units from the database (optionally filtered by specific properties).

```
Eg: units = [Unit1, Unit2, Unit3, Unit4, Unit5]
```

Step 2: Count the total number of units.

```
Eg: total_units = 5
```

Step 3: Filter units with status = 'occupied' and count them.

```
Eg: occupied_units = [Unit1, Unit3, Unit4] → count = 3
```

Step 4: Handle edge case where total_units is 0 to avoid division by zero.

```
Eg: if total_units == 0, occupancy_rate = 0.0
```

Step 5: Calculate occupancy rate using the formula: $(\text{occupied_units} / \text{total_units}) \times 100$

```
Eg: occupancy_rate = (3 / 5) × 100 = 60.0
```

Step 6: Round the occupancy rate to 2 decimal places for readability.

```
Eg: occupancy_rate = 60.00
```

Step 7: Categorize occupancy status based on the calculated rate:

- If occupancy_rate ≥ 80: "High"
- If occupancy_rate ≥ 50: "Medium"
- If occupancy_rate < 50: "Low"

```
Eg: occupancy_rate = 60.00, so status = "Medium"
```

Step 8: Return the result dictionary with all calculated values.

```
Eg: {  
  'total_units': 5,  
  'occupied_units': 3,  
  'occupancy_rate': 60.00,  
  'status': 'Medium'  
}
```

3.3.3 Maintenance Request Prioritization Algorithm

In this system, the maintenance request prioritization algorithm is used to assign priority levels (High, Medium, Low) to maintenance requests based on their urgency. The algorithm considers two factors: the age of the request (how many days since creation) and the current status. This helps property owners and maintenance staff prioritize which requests need immediate attention.

The steps involved in the implementation of the algorithm are:

Step 1: Initialize `priority_score` to 0.

Eg: `priority_score = 0`

Step 2: Calculate the age of the maintenance request in days.

Eg: `created_at = 2024-01-15, today = 2024-01-25`
`days_old = 2024-01-25 - 2024-01-15 = 10 days`

Step 3: Apply age factor scoring based on `days_old`:

- If `days_old > 7`: add 3 points
- If `days_old > 3`: add 2 points
- Else: add 1 point

Eg: `days_old = 10`, so `priority_score += 3` → `priority_score = 3`

Step 4: Apply status factor scoring based on current status:

- If `status == 'pending'`: add 2 points
- If `status == 'in_progress'`: add 1 point
- Else: add 0 points

Eg: `status = 'pending'`, so `priority_score += 2` → `priority_score = 5`

Step 5: Determine final priority level based on total `priority_score`:

- If `priority_score ≥ 4`: "High"
- If `priority_score ≥ 3`: "Medium"
- Else: "Low"

Eg: `priority_score = 5`, so `priority = "High"`

Step 6: Return the priority level.

Eg: Final `priority = "High"`

3.4 Result Analysis

This table shows the result analysis of all three algorithms used in the Property Management System:

Table 3.1: Result analysis of algorithms used in the Property Management System

Algorithm	Efficiency	Time Complexity	Conclusion
Late Fee Calculation	Late Fee Calculation uses a simple conditional structure to determine penalty fees based on days overdue. The algorithm performs constant-time operations with simple arithmetic and comparisons. It is highly efficient and suitable for real-time payment processing.	Best Case: $O(1)$ Average/Worst Case: $O(1)$	All three algorithms significantly improved the Property Management System's functionality. Late Fee Calculation provides instant penalty computation, Occupancy Rate Calculation offers real-time property utilization insights, and Maintenance Request Prioritization enables efficient workflow management. Among the three, Late Fee Calculation is the most efficient due to its constant time complexity, while Maintenance Request Prioritization provides the most value for operational efficiency.
Occupancy Rate Calculation	Occupancy Rate Calculation performs database queries to count total and occupied units, then applies simple arithmetic for percentage calculation and categorization. The algorithm efficiently handles property aggregation and provides instant occupancy metrics for decision-making.	Best Case: $O(n)$ Average/Worst Case: $O(n)$ Where n = number of units	
Maintenance Request Prioritization	Maintenance Request Prioritization uses a simple additive scoring system combining age and status factors. The algorithm performs constant-time calculations per request and is suitable for real-time priority assignment. It enables efficient maintenance workflow	Best Case: $O(1)$ Average/Worst Case: $O(1)$ Per request	

Algorithm	Efficiency	Time Complexity	Conclusion
	management without complex computations.		

Summary:

- **Late Fee Calculation** is the most efficient with $O(1)$ complexity, making it ideal for high-volume payment processing
- **Occupancy Rate Calculation** has $O(n)$ complexity due to database queries but remains efficient for typical property management datasets
- **Maintenance Request Prioritization** has $O(1)$ complexity per request, making it highly scalable for large maintenance request volumes

All algorithms are deterministic, easy to implement, and provide clear business value for the Property Management System.

CHAPTER 4: IMPLEMENTATION AND TESTING

4.1 Implementation

4.1.1 Tools Used

i. Front-End Tools

- **Next.js:** Used to build the Property Management System with efficient routing, server-side rendering, and API handling for properties, units, leases, and payments.
- **Tailwind CSS:** Used for responsive and consistent UI design with a modern, utility-first approach.
- **JavaScript (React):** Handles interactivity such as form validation, dynamic updates, and state management.

ii. Back-End Tools

- **Python:** Python is used as the core programming language to implement the system logic such as user authentication, data processing, attendance handling, assignment management, result calculation, notification handling, and algorithm implementation. Various built-in and external Python libraries are used in the system, including re for regular expression validation, random for username and password generation, json for data handling, datetime and timezone for date and time operations, and Django ORM utilities such as Q, Sum, Avg, and Count for database querying and analysis. Additional libraries such as messages are used for displaying system notifications and validation messages.
- **Django Framework:** Django is used to build the backend structure of the system. It handles models (database structure), views (business logic), and templates (data rendering). Django functionalities such as render, redirect, get_object_or_404, IntegrityError, form handling, file uploading, and ORM-based database operations are extensively used to develop and manage the different modules of the School Management System efficiently.

iii. Database Tools

- **MySQL:** The database is used to store all system data such as user details, property, tenant, lease, maintenance, and notifications.

iv. Design Tools

- **Draw.io:** Used to create system diagrams such as DFDs and ER diagrams for visualizing system architecture and database relationships.

4.1.2 Implementation Details of Modules

Different modules of PROPERTY MANAGEMENT SYSTEM are:

i. Login/Signup Module

The Login/Signup module enables users to securely access the system by providing their email and password as input fields. After entering valid credentials, users are authenticated against the database, and only authorized users are allowed to log in. If the credentials are incorrect, the system displays appropriate error messages to prevent unauthorized access. Once successfully authenticated, users are redirected to their respective dashboards based on their roles, such as admin, property owner, or tenant. This module also supports role-based access control, ensuring that each user can only access the features and functionalities assigned to their role, thereby maintaining system security and proper data management.

```
class LoginAPIView(APIView): 2 usages
    authentication_classes = []
    permission_classes = []

    def post(self, request):
        username = request.data.get("username")
        password = request.data.get("password")
        user = authenticate(username=username, password=password)
        if user is None:
            return Response( data: {"detail": "Invalid credentials"},
                             status=status.HTTP_400_BAD_REQUEST)
        refresh = RefreshToken.for_user(user)
        return Response({
            "access": str(refresh.access_token),
            "refresh": str(refresh),
            "user": UserSerializer(user).data
        })
```

ii. Property Management Module

The Property Module allows the admin or property owner to manage property-related information within the system. It includes input fields such as property name, property type, location, rent amount, number of rooms, and property status to capture complete property details. Through this module, authorized users can add new properties, update existing property information, and delete properties when necessary. The system ensures that all property records are properly maintained and available for further operations such as unit and lease management, maintenance tracking, and payment recording.

```

class PropertyListCreateAPIView(APIView): 2 usages
    permission_classes = [IsAuthenticated, IsAdminOrOwner]

    def get_queryset(self, user):
        if user.role == "admin":
            return Property.objects.all().order_by("-created_at")
        return Property.objects.filter(owner=user).order_by("-created_at")

    def get(self, request):
        paginator = SimplePagination()
        queryset = self.get_queryset(request.user)
        page = paginator.paginate_queryset(queryset, request)
        serializer = PropertySerializer(page, many=True)
        return paginator.get_paginated_response(serializer.data)

    def post(self, request):
        serializer = PropertySerializer(data=request.data)
        serializer.is_valid(raise_exception=True)
        property_obj = serializer.save(owner=request.user)
        return Response(PropertySerializer(property_obj).data, status=status.HTTP_201_CREATED)

```

```

class PropertyDetailAPIView(APIView): 2 usages
    permission_classes = [IsAuthenticated, IsAdminOrOwner]

    def get_object(self, request, pk):
        queryset = Property.objects.all() if request.user.role == "admin" else
            Property.objects.filter(owner=request.user)
        return queryset.filter(pk=pk).first()

    def get(self, request, pk):
        property_obj = self.get_object(request, pk)
        if not property_obj:
            return Response(data={"detail": "Not found"}, status=status.HTTP_404_NOT_FOUND)
        return Response(PropertySerializer(property_obj).data)

    def patch(self, request, pk):
        property_obj = self.get_object(request, pk)
        if not property_obj:
            return Response(data={"detail": "Not found"}, status=status.HTTP_404_NOT_FOUND)
        serializer = PropertySerializer(property_obj, data=request.data, partial=True)
        serializer.is_valid(raise_exception=True)
        serializer.save()
        return Response(serializer.data)

    def delete(self, request, pk):
        property_obj = self.get_object(request, pk)
        if not property_obj:
            return Response(data={"detail": "Not found"}, status=status.HTTP_404_NOT_FOUND)
        property_obj.delete()
        return Response(status=status.HTTP_204_NO_CONTENT)

```


iii. Payment Management Module

The Payment Module enables the system to manage rent payment efficiently. It includes input fields such as tenant name, property, payment amount, payment date, and payment status to record transaction details accurately. Through this module, tenants can make payments, while admins or property owners can view, verify, and track payment records. The module also helps in maintaining payment history and generating payment reports, ensuring transparency and proper financial management within the system.

```
class PaymentListAPIView(APIView): 2 usages
    permission_classes = [IsAuthenticated]

    def get_queryset(self, user):
        base = Payment.objects.select_related("lease", "lease__unit",
            "lease__unit__property").prefetch_related("lease__tenants")
        if user.role == "tenant":
            return base.filter(lease__tenants=user).order_by("-due_date")
        if user.role == "admin":
            return base.order_by("-due_date")
        return base.filter(lease__unit__property__owner=user).order_by("-due_date")

    def get(self, request):
        queryset = self.get_queryset(request.user)
        tenant_id = request.query_params.get("tenant_id")
        lease_id = request.query_params.get("lease_id")
        if tenant_id:
            queryset = queryset.filter(lease__tenants__id=tenant_id)
        if lease_id:
            queryset = queryset.filter(lease_id=lease_id)
        paginator = SimplePagination()
        page = paginator.paginate_queryset(queryset.distinct(), request)
        serializer = PaymentSerializer(page, many=True)
        return paginator.get_paginated_response(serializer.data)
```

iv. Maintenance Management Module

The Maintenance Module allows tenants and admins to manage maintenance requests for properties. It includes input fields such as property name, issue title, description, request date, and maintenance status to record maintenance-related details. Tenants can submit maintenance requests whenever issues arise, while admins or property managers can review, assign, update, and resolve the requests accordingly. The module helps in tracking the progress of maintenance tasks and ensures timely resolution of property-related issues, improving tenant satisfaction and property management efficiency.

class MaintenanceListCreateAPIView(APIView): 2 usages

⚠️ 3 ⚠️ 3 ^

```
permission_classes = [IsAuthenticated]
```

```
def get_queryset(self, user):
```

```
    base = MaintenanceRequest.objects.select_related("tenant", "unit",
        "unit__property").order_by("-created_at")
```

```
    if user.role == "tenant":
```

```
        return base.filter(tenant=user)
```

```
    if user.role == "admin":
```

```
        return base
```

```
    return base.filter(unit__property__owner=user)
```

```
def get(self, request):
```

```
    queryset = self.get_queryset(request.user)
```

```
    paginator = SimplePagination()
```

```
    page = paginator.paginate_queryset(queryset, request)
```

```
    serializer = MaintenanceRequestSerializer(page, many=True)
```

```
    return paginator.get_paginated_response(serializer.data)
```

```
def post(self, request):
```

```
    if request.user.role != "tenant":
```

```
        return Response( data: {"detail": "Only tenants can create requests"},
```

```
            status=status.HTTP_403_FORBIDDEN)
```

```
    active_lease = Lease.objects.filter(
```

```
        tenants=request.user,
```

```
        start_date__lte=timezone.now().date(),
```

```
        end_date__gte=timezone.now().date(),
```

```
    ).select_related("unit").first()
```

```
    if not active_lease:
```

```
        return Response( data: {"detail": "No active lease found for tenant."},
```

```
            status=status.HTTP_400_BAD_REQUEST)
```

```
    payload = request.data.copy()
```

```
    payload["unit"] = active_lease.unit_id
```

```
    serializer = MaintenanceRequestSerializer(data=payload)
```

```
    serializer.is_valid(raise_exception=True)
```

```
    maintenance = serializer.save(tenant=request.user, unit=active_lease.unit,
```

```
        status="pending")
```

v. Lease Management Module

The Lease Module enables the admin or property owner to manage lease agreements between tenants and properties. It includes input fields such as tenant name, property name, lease start date, lease end date, monthly rent, and lease status to store complete lease details. Through this module, authorized users can create lease agreements. The system also helps in tracking lease durations and renewal dates, ensuring proper management of rental agreements and reducing the risk of expired or unmanaged leases.

```

class LeaseDetailAPIView(APIView): 2 usages
    permission_classes = [IsAuthenticated, IsAdminOrOwner]

    def get_object(self, request, pk):
        queryset = Lease.objects.all() if request.user.role == "admin" else
            Lease.objects.filter(unit__property__owner=request.user)
        return queryset.filter(pk=pk).first()

    def get(self, request, pk):
        lease = self.get_object(request, pk)
        if not lease:
            return Response( data: {"detail": "Not found"}, status=status.HTTP_404_NOT_FOUND)
        return Response(LeaseSerializer(lease).data)

    def patch(self, request, pk):
        lease = self.get_object(request, pk)
        if not lease:
            return Response( data: {"detail": "Not found"}, status=status.HTTP_404_NOT_FOUND)
        serializer = LeaseSerializer(lease, data=request.data, partial=True)
        serializer.is_valid(raise_exception=True)
        tenant_ids = serializer.validated_data.pop("tenant_ids", None)
        lease = serializer.save()
        if tenant_ids is not None:
            lease.tenants.set(tenant_ids)
        lease.unit.status = "occupied" if lease.tenants.exists() else "vacant"
        lease.unit.save(update_fields=["status"])
        return Response(LeaseSerializer(lease).data)

    def delete(self, request, pk):
        lease = self.get_object(request, pk)
        if not lease:
            return Response( data: {"detail": "Not found"}, status=status.HTTP_404_NOT_FOUND)
        unit = lease.unit
        lease.delete()
        if not LeaseTenant.objects.filter(lease__unit=unit).exists():
            unit.status = "vacant"
            unit.save(update_fields=["status"])
        return Response(status=status.HTTP_204_NO_CONTENT)

```

4.2 Testing

Testing part includes making sure all features do what they are supposed to, that the system is easy to use, secure, and performs well. It's done to find and fix problems before the system is used by others.

4.2.1 Test cases for Unit Testing

Table 4.1: Test Case for Login

Test case No.	Test Case	Expected results	Status
1	Logging into website	Email and password provided correct	Successful
3	Logging into website	Password Incorrect	Unsuccessful
4	Logging into website	Any field left empty	Unsuccessful

Table 4.2: Test Cases for Sign Up

Test cases No.	Test Case	Expected results	Status
1	Registration for new user	All details provided correctly	Successful
3	Registration for new user	Any field left empty	Unsuccessful

Table 4.3: Test Cases for Creating Property

Test cases No.	Test Case	Expected results	Status
1	Property	All details provided correctly	Successful
2	Unit	One field empty	Unsuccessful
3	Lease	Prevent more lease assignment than unit accomodation	Unsuccessful

4.2.2 Test Cases for System Testing

Table 4.5 Test Cases for system testing of the PROPERTY MANAGEMENT
SYSTEM

Test Case ID	Test Name	Test case Details	Expected Result	Test Result	Status
1	Application Launch Test	Open the application	System Login Page opens	System Login Page opens	Pass
2	Login Functionality Test	Provide Correct login credentials	Login successful	Login successful	Pass
3	Add Property Functionality Test	Click on add property and enter details	Property added successfully	Property added successfully	Pass
4	Add Lease Functionality Test	Select tenant and add lease	Lease deleted successfully	Lease deleted successfully	Pass
5	Add Maintenance Ticket	Create a maintenance ticket	Maintenance ticket created	Maintenance ticket created successfully	Pass
6	Update Payment Status	Change the status to paid	Status changes from pending to paid	Payment is paid	Pass
7	Logout functionality Test	Click on logout button	System Login page opens	System login page opens	Pass

CHAPTER 5: CONCLUSIONS AND FUTURE RECOMMENDATIONS

Lesson Learnt/Outcome

During the development of PropFlow: A Web-Based Property Management System, I gained strong practical knowledge of full-stack web application development using Django, Python, Next, and PostgreSQL. I learned how different modules such as property management, tenant management, rent tracking, maintenance request handling, occupancy management, and dashboard reporting are designed, implemented, and integrated to form a complete system. This project helped me understand the importance of database management, session management, role-based authentication, and data validation for building secure and reliable web applications. I also improved my problem-solving skills by handling real-world issues such as duplicate rent records, tenant data consistency, maintenance request tracking, and system errors. The project further enhanced my understanding of modular system development using the Hybrid Incremental Model, where each module was developed and tested independently before integration. Overall, this project enhanced my technical skills and provided valuable experience in developing a property management system for property owners managing multiple rental units.

Conclusion

PropFlow: A Web-Based Property Management System was successfully designed and implemented to digitalize and simplify property management activities such as property listing, tenant management, rent tracking, maintenance handling, occupancy tracking, and reporting. The system provides a user-friendly and centralized platform that allows property owners to efficiently manage multiple properties and units from a single system. By integrating essential modules into one platform, the system reduces manual workload, minimizes record-keeping errors, improves data organization, and provides better visibility into property operations. The system also supports real-time tracking of tenant information, payment status, maintenance requests, and unit availability, helping property owners manage their operations more effectively. Overall, the project achieved its primary objective of creating a structured, reliable, and efficient web-based solution for property management.

Future recommendations

In the future, PropFlow can be further improved by adding more advanced features to enhance functionality, scalability, and user experience. A mobile application version can be developed to allow property owners and tenants to access the system more conveniently through smartphones and tablets. The system can also be integrated with SMS or email notification services to provide real-time alerts regarding rent due dates, payment confirmations, maintenance request updates, and property announcements. Online payment gateway integration can be implemented to support secure and automated digital rent payments. Advanced analytics and AI-based prediction features can also be added to provide insights into occupancy trends, rent collection patterns, and property performance. Furthermore, additional modules such as lease agreement management, visitor management, and document management can be incorporated to make the system more comprehensive. Cloud-based backup services and enhanced security mechanisms can also be implemented to improve reliability, scalability, and data protection in future versions of the system.

References

- [1] K. Acharya, “Online Property Management System Project Report,” ResearchGate. [Online]. Available: <https://www.researchgate.net/>

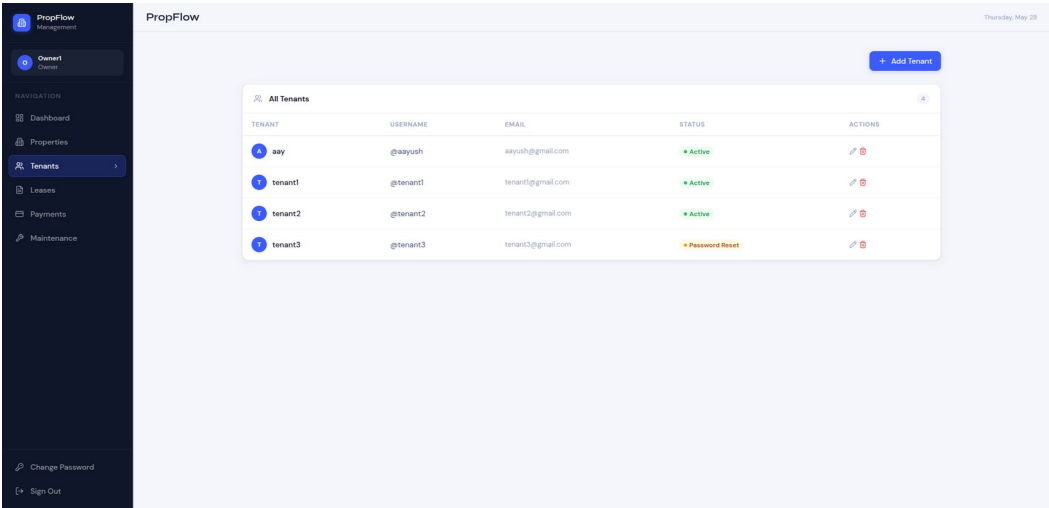
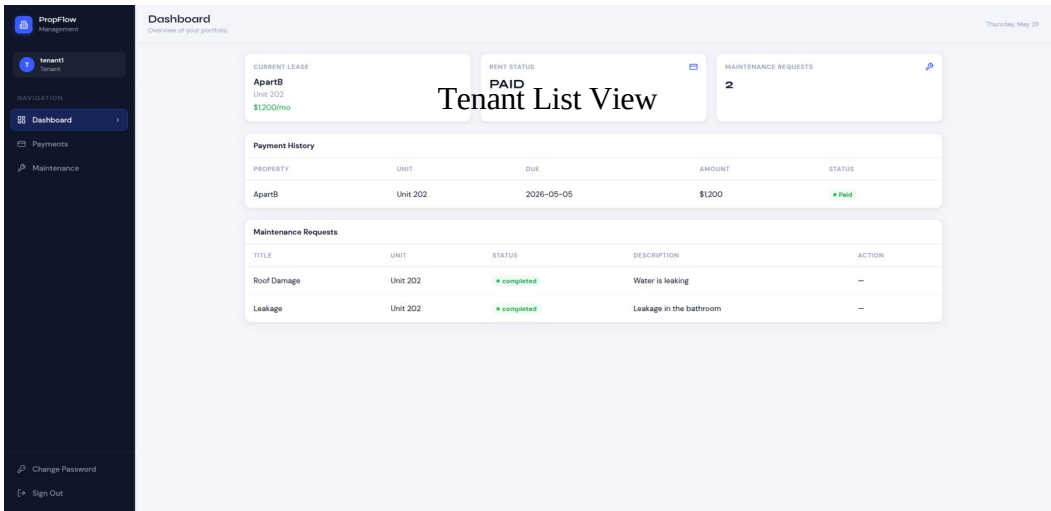
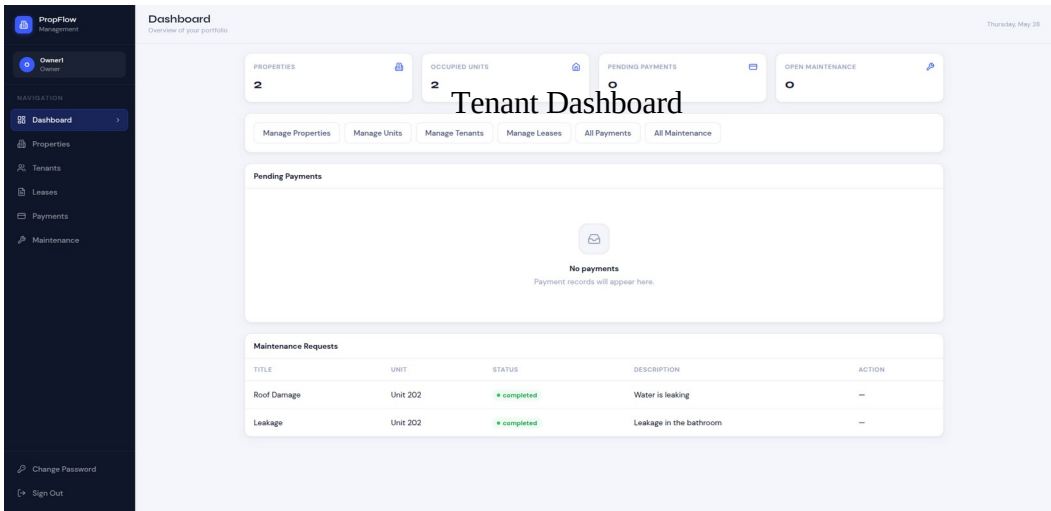
- [2] “Types of Feasibility Study in Software Project Development,” GeeksforGeeks. [Online]. Available: <https://www.geeksforgeeks.org/types-of-feasibility-study-in-software-project-development/>

- [3] “Next.js Documentation,” Vercel. [Online]. Available: <https://nextjs.org/docs>

- [4] “Django Documentation,” Django Software Foundation. [Online]. Available: <https://docs.djangoproject.com/>

- [5] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Pearson, 2016.

APPENDIX



PropFlow
Management

Owner1
Owner

NAVIGATION

Dashboard

Properties

Tenants

Leases

Payments

Maintenance

Change Password

Sign Out

Payments

Payment history & records

Thursday, May 28

\$ Total collected \$5,400.00

Payment History

3 records

TENANT	PROPERTY	UNIT	AMOUNT	DATE	STATUS	ACTION
tenant2	ApartB	Unit 203	\$1,400.00	May 4, 2026	paid	—
tenant1	ApartB	Unit 202	\$1,200.00	May 3, 2026	paid	—
tenant2	ApartB	Unit 203	\$2,800.00	May 4, 2026	paid	—

Payment View

PropFlow
Management

tenant1
tenant

NAVIGATION

Dashboard

Payments

Maintenance

Change Password

Sign Out

Maintenance

Service requests & status

Thursday, May 28

Pending

In Progress

Completed 2

Cancelled

Create Ticket

Maintenance Requests

All (2) Pending (0) In Progress (0) Completed (2)

TENANT	UNIT	ISSUE TYPE	DESCRIPTION	SUBMITTED	STATUS	ACTION
tenant1	Unit 202	Roof Damage	Water is leaking	May 4, 2026	Completed	Completed
tenant1	Unit 202	Leakage	Leakage in the bathroom	May 4, 2026	Completed	Completed

Maintenance View